

Java

Exception Handling in Java :-

What is Exception :-

- An exception is an unexpected event in a program.
- It occurs during program execution.
- It breaks the normal flow of the program.
- In Java, it is an object representing an error.

What is Exception Handling :-

- Exception handling in Java is a process used to handle runtime errors.
- It prevents program termination when an error occurs.
- It uses try, catch, finally, throw, and throws keywords.
- This mechanism helps maintain normal program flow.

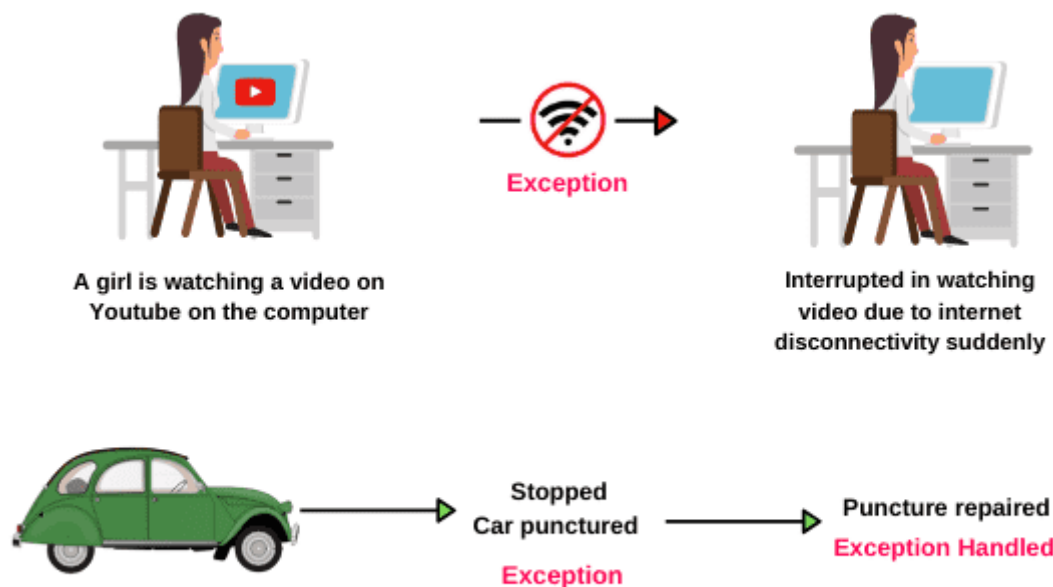
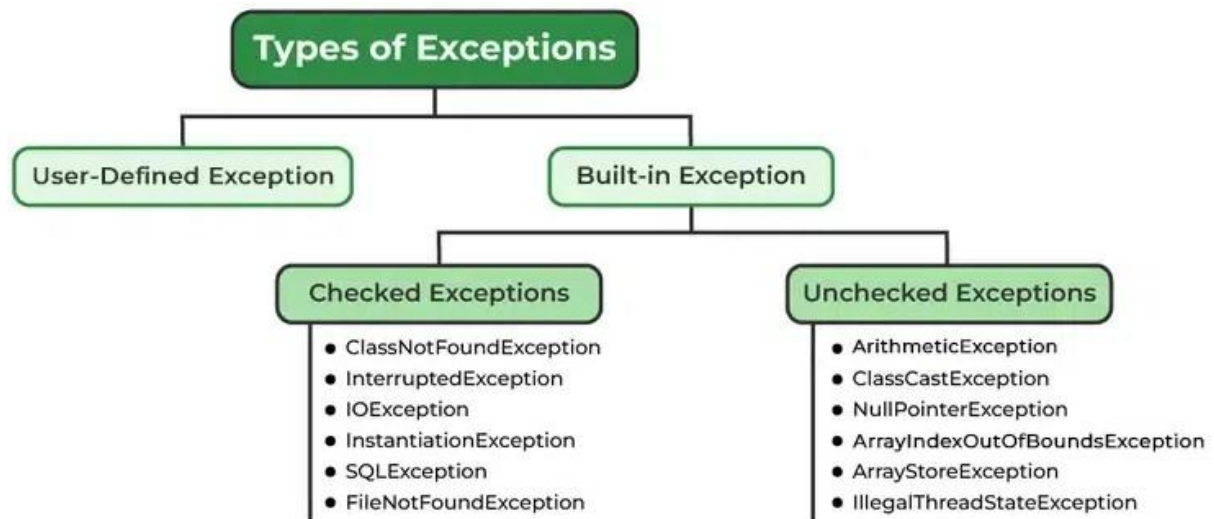


Fig: Realtime Example of Exception Handling

Types of Exceptions :-

- There are mainly two types of exceptions: checked and unchecked.
 - An error is considered an unchecked exception.
 - According to Oracle, there are three types:
1. Checked Exception
 2. Unchecked Exception
 3. Error



1. Checked Exception :-

- Checked exceptions are detected at compile-time.
- They must be handled using try-catch or declared with throws.
- Example:
 1. IOException
 2. SQLException

2. Unchecked Exception :-

- Unchecked exceptions occur at runtime.
- Handling is optional, but recommended for safe execution.
- Example:
 1. ArithmeticException
 2. NullPointerException

3. Error :-

- Errors are serious problems that the program usually cannot handle.
- They are generated by the JVM, not by the application.
- Example:
 3. OutOfMemoryError,
 4. StackOverflowError.

To Handle the Exception using the Keywords :-

1. try

- The try block contains code that might cause an exception.
- If an exception occurs, it is thrown to the corresponding catch block.
- The code inside try is monitored for errors during execution.
- A try block must be followed by at least one catch or finally block.

2. catch

- The catch block is used to handle exceptions thrown by the try block.
- It specifies the type of exception it can handle.
- If no exception occurs, the catch block is skipped.
- It ensures the program continues running without crashing.

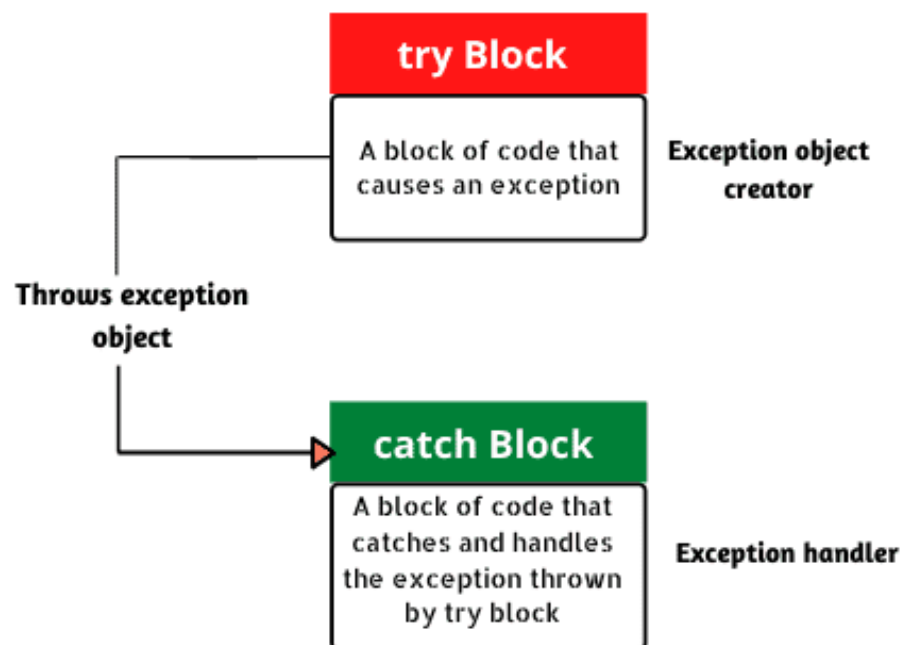


Fig: Exception handling mechanism

Implementation of Exceptions :-

Example – Arithmetic Exception

```
package exception_handling;

public class handling_arithmetic_exception {
    public static void main(String[] args) {
        try
        {
            int num=9/0;
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
        System.out.println("Tuesday");
    }
}
```

Output –

[java.lang.ArithmeticException](#): / by zero
Tuesday

Example – ArrayIndexOutOfBoundsException Exception

```
package exception_handling;

public class char_exp {

    public static void main(String[] args) {
        char ch[]=new char[2];
        ch[0]='A';
        ch[1]='*';
        try
        {
            ch[2]='0';
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Output –

[java.lang.ArrayIndexOutOfBoundsException](#): Index 2 out of bounds for length 2

Example –

```
package exception_handling;

public class str_handling {

    public static void main(String[] args) {
        String str=null;
        try
        {
            System.out.println(str.length());
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

Output –

[java.lang.NullPointerException](#): Cannot invoke "String.length()" because "str" is null

Difference Between Errors & Exceptions :-

Errors	Exceptions
Recovering from Error is not possible.	We can recover from exceptions by either using try-catch block or throwing exceptions back to the caller.
All errors in java are unchecked type.	Exceptions include both checked as well as unchecked type.
Errors are mostly caused by the environment in which program is running.	Program itself is responsible for causing exceptions.
Errors can occur at compile time.	Unchecked exceptions occur at runtime whereas checked exceptions occur at compile time
They are defined in java.lang.Error package.	They are defined in java.lang.Exception package
Examples : java.lang.StackOverflowError, java.lang.OutOfMemoryError	Examples : Checked Exceptions : SQLException, IOException Unchecked Exceptions : ArrayIndexOutOfBoundsException, NullPointerException, ArithmeticException.

Difference Between Checked Exception & Unchecked Exception :-

Checked Exception	Unchecked Exception
Checked exceptions are checked at compile time.	Unchecked exceptions are checked at run time.
Derived from Exception.	Derived from RuntimeException.
Caused by external factors like file I/O and database connection cause the checked Exception.	Caused by programming bugs like logical errors cause unchecked Exceptions.
Checked exceptions must be handled using a try-catch block or must be declared using the throws keyword	No handling is required.
Examples: IOException, SQLException, FileNotFoundException.	Examples: NullPointerException, ArrayIndexOutOfBoundsException.